

Reflection Template

Use Case	UC-13
Use Case Name	Add Comment to Task
Requirement IDs	FR16, FR19, FR20, FR21, FR22, FR23
Matching Feature Comparison Sets	CS-26, CS-27, CS-28
Implementation Order	Java first -> Kotlin second

Feature Relevance

Feature/s:

Data classes/records, Null safety, extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, the Comment entity introduced new boilerplate in Java which is now more routine. The Elvis operator appeared naturally in the mapper for null handling. The extension function was reused without changes.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	5
Compactness of logic	5	5
Perceived implementation effort	4	5

Justifications

Readability:

Kotlin is still more readable due to the compact entity definition. Java's entity with getters, setters and constructors is more verbose but familiar by now.

Compactness of logic:

Both are equally compact at this point. The Comment entity is simpler than Task with fewer fields, so the amount of code difference is less impactful than in UC6.

Implementation effort:

Java required more lines but the pattern is now routine. Kotlin still felt slightly faster overall.

Observed structural differences

The Comment entity in Java requires getters, setters, and constructors, same pattern as UC6 but with fewer fields. Kotlin remains more compact.

The mapper uses ?: in Kotlin for null safety on id, task.id, and author.id. Java has no equivalent compile-time signal.

Extension function vs RepositoryUtils, consistent pattern, no new differences.

Final comparative summary

UC13 confirmed existing patterns. The entity boilerplate gap between Java and Kotlin remains but feels less significant as the pattern becomes routine. Null safety in the mapper and the extension function continue to show consistent differences across use cases.